

# Metabolomics Data Analysis Workshop 2026

Paul Stewart

2026-07-29

## Introduction

### Set up RStudio

Before we start writing or editing code, there are a few settings in RStudio that can make your experience much more comfortable. One helpful option is to turn on Soft Wrap, so that long lines of code or text automatically wrap within the editor window instead of scrolling off to the right. You can enable this by going to Tools -> Global Options -> Code -> Editing and checking Soft-wrap source files.

Another useful feature is Rainbow Parentheses, which color-matches pairs of parentheses and brackets to help you keep track of nesting. This is especially helpful when working with complex expressions. You can turn this on under Tools -> Global Options -> Code -> Display by checking Rainbow parentheses.

Finally, you may want to change the background color of the RStudio editor to something easier on the eyes. To do this, go to Tools -> Global Options -> Appearance and choose a different editor theme, such as “Tomorrow Night” for a dark background or “Textmate” for a clean, light look.

### R Markdown

This is an R Markdown file (or the PDF report generated from it, depending on what you are reading). R Markdown lets us combine regular text and executable code in a single document. You can run code chunks interactively or use the Knit button to create a formatted report (such as a PDF, HTML, or Word document). We will focus first on running code interactively and touch on reports towards the end. For the most part, code run through an R Markdown file are equivalent to them being run in the console.

In an R Markdown file, we use text characters to format other text that appear once the document is rendered into the formatted report.

- Lines that begin with `#` are section headers.
- `#` creates a main section
- `##` creates a subsection
- `*`'s around some text will *italicize* it
- A single `-` followed by a space will make a bulleted list

Code is written in code chunks, which are formatted like this:

```
21 + 21
```

```
## [1] 42
```

You can run chunks by pressing the green arrow on the right hand side of the chunk, by pressing `Cmd + Enter` on a Mac or `Ctrl + Enter` on Windows. When running a chunk, the output appears underneath the chunk.

## Read in and view data

### Set the working directory

It is important that R knows where to look for your files. This is controlled by the working directory. If R is giving you an error about a file not existing but you are sure it does, then the most common problem is that the working directory was not set correctly (second most common is that there is a typo in your file location).

If we are using an R Project (which we are), the working directory is automatically set to the folder that contains the .Rproj file. However, since we are working in an R Markdown document, it may default the working directory to the location of the .Rmd file instead. This can be confusing, especially when code behaves differently depending on if it is run in the console or inside chunks.

To fix this, we will explicitly set the working directory using the here package. This ensures that all file paths are interpreted relative to the folder where the R Project is located, rather than where the .Rmd file happens to live.

To install the here package, run this command in the console. The chunk below has the option eval = FALSE, which means it will not be evaluated when we knit the document or run all chunks at once. However, we can still run it manually using the green arrow or by placing your cursor in the chunk and pressing Ctrl + Enter (or Cmd + Enter on Mac).

```
install.packages("here")
```

By default, this installs the package from CRAN, the Comprehensive R Archive Network (<https://cran.r-project.org>), which is the main repository for R packages (in addition to hosting the R package and source code).

Once it is installed, load the here package using

```
library(here)
```

```
## here() starts at /Users/paulstewart/Library/CloudStorage/Box-Box/projects/metabolomics_workshop_2026
```

We will now add a setup chunk that sets the root directory for the entire document. It is important that this chunk option has “setup” included for this to work:

```
knitr::opts_knit$set(root.dir = here::here())
```

### Experimental data

Now let us load the metabolite abundance data. We give it a short, descriptive variable name like metab so we know what the contents are without having to look at the contents. Header = TRUE means that the first row will be used for the column names, and row.names = 1 indicates that the first row will be used for the row names:

```
metab = read.delim("data/metabolite_abundance_2026-07-28.txt",  
  header = TRUE, row.names = 1)
```

Let us also load the sample meta data (subsequently referred to as “sample\_meta” for shorthand):

```
sample_meta = read.delim("data/sample_meta_2026-07-28.txt", header = TRUE)
```

We can look at the first few rows with the head() function. If you are not sure what a function does (or if you forget), try running ?function\_name in the console.

```
head(metab)
```

```
##                X4010.Dish.0.cGy.sGCR X4015.Dish.0.cGy.sGCR  
## 7-Methylguanine                2769164.5                1368913.2  
## 8-oxoG                        246277.7                  323311.2
```

|                       |                          |                          |
|-----------------------|--------------------------|--------------------------|
| ## Succinic acid      | 6639808.8                | 1308332.8                |
| ## UDP-Hexose         | 8713523.5                | 3465812.5                |
| ## N2-Acetylornithine | 3939274.4                | 5040621.4                |
| ## Cytidine           | 35182976.7               | 16661250.6               |
| ##                    | X4026.Dish.0.cGy.sGCR    | X4104.Dish.0.cGy.sGCR    |
| ## 7-Methylguanine    | 1065909.6                | 1165668.6                |
| ## 8-oxoG             | 320123.1                 | 155842.3                 |
| ## Succinic acid      | 3381226.0                | 4459418.9                |
| ## UDP-Hexose         | 7969206.4                | 7783445.5                |
| ## N2-Acetylornithine | 6288896.4                | 2967679.9                |
| ## Cytidine           | 20454134.6               | 27278216.5               |
| ##                    | X4106.Dish.0.cGy.sGCR    | X4181.Dish.0.cGy.sGCR    |
| ## 7-Methylguanine    | 1039749.8                | 1013177.5                |
| ## 8-oxoG             | 243354.4                 | 198652.8                 |
| ## Succinic acid      | 4671689.9                | 6479653.7                |
| ## UDP-Hexose         | 11324223.7               | 12920904.2               |
| ## N2-Acetylornithine | 5928247.3                | 5083954.0                |
| ## Cytidine           | 18849889.1               | 27656343.2               |
| ##                    | X4010.Dish.50.cGy.sGCR   | X4015.Dish.50.cGy.sGCR   |
| ## 7-Methylguanine    | 1926916.3                | 1119515.3                |
| ## 8-oxoG             | 149197.2                 | 205922.1                 |
| ## Succinic acid      | 3160987.6                | 2836311.5                |
| ## UDP-Hexose         | 6605160.9                | 7547374.7                |
| ## N2-Acetylornithine | 2455470.8                | 3292519.1                |
| ## Cytidine           | 31682522.9               | 20032669.4               |
| ##                    | X4026.Dish.50.cGy.sGCR   | X4104.Dish.50.cGy.sGCR   |
| ## 7-Methylguanine    | 678706.2                 | 1210638.2                |
| ## 8-oxoG             | 150448.2                 | 192363.2                 |
| ## Succinic acid      | 2518031.3                | 1901065.0                |
| ## UDP-Hexose         | 4560993.3                | 4520346.4                |
| ## N2-Acetylornithine | 3227460.0                | 3521069.2                |
| ## Cytidine           | 19730633.0               | 22656353.6               |
| ##                    | X4106.Dish.50.cGy.sGCR   | X4181.Dish.50.cGy.sGCR   |
| ## 7-Methylguanine    | 1008742.6                | 1154117.9                |
| ## 8-oxoG             | 147589.9                 | 169534.6                 |
| ## Succinic acid      | 2442808.7                | 3600233.3                |
| ## UDP-Hexose         | 6015905.9                | 12716144.4               |
| ## N2-Acetylornithine | 2742450.4                | 5030812.0                |
| ## Cytidine           | 17736077.2               | 23177273.2               |
| ##                    | X4010.Dish.100.cGy.Gamma | X4015.Dish.100.cGy.Gamma |
| ## 7-Methylguanine    | 1650095.3                | 1055828.9                |
| ## 8-oxoG             | 210190.9                 | 179170.6                 |
| ## Succinic acid      | 4005127.2                | 2054376.3                |
| ## UDP-Hexose         | 7651951.2                | 5399451.9                |
| ## N2-Acetylornithine | 5566114.9                | 2169086.9                |
| ## Cytidine           | 37359501.8               | 24834883.5               |
| ##                    | X4026.Dish.100.cGy.Gamma | X4104.Dish.100.cGy.Gamma |
| ## 7-Methylguanine    | 752840.0                 | 896385.5                 |
| ## 8-oxoG             | 149312.2                 | 158024.7                 |
| ## Succinic acid      | 4012117.3                | 1723168.6                |
| ## UDP-Hexose         | 5474190.5                | 3387421.6                |
| ## N2-Acetylornithine | 2284557.6                | 2202814.2                |
| ## Cytidine           | 20716143.9               | 23480154.8               |
| ##                    | X4106.Dish.100.cGy.Gamma | X4181.Dish.100.cGy.Gamma |

|                       |                              |                              |
|-----------------------|------------------------------|------------------------------|
| ## 7-Methylguanine    | 946172.7                     | 973979.7                     |
| ## 8-oxoG             | 176401.9                     | 200370.5                     |
| ## Succinic acid      | 2077958.0                    | 2353969.8                    |
| ## UDP-Hexose         | 8783617.0                    | 6558440.5                    |
| ## N2-Acetylornithine | 4065620.9                    | 3487138.4                    |
| ## Cytidine           | 22190068.3                   | 22206038.8                   |
| ##                    | X4010.Rotation.0.cGy.sGCR    | X4015.Rotation.0.cGy.sGCR    |
| ## 7-Methylguanine    | 1112591.6                    | 834813.2                     |
| ## 8-oxoG             | 143723.8                     | 258630.7                     |
| ## Succinic acid      | 2315446.0                    | 3491221.0                    |
| ## UDP-Hexose         | 12543558.3                   | 8601999.0                    |
| ## N2-Acetylornithine | 2321838.1                    | 5206991.2                    |
| ## Cytidine           | 16445435.5                   | 16428907.1                   |
| ##                    | X4026.Rotation.0.cGy.sGCR    | X4104.Rotation.0.cGy.sGCR    |
| ## 7-Methylguanine    | 581899.2                     | 1600690.4                    |
| ## 8-oxoG             | 244280.7                     | 246522.5                     |
| ## Succinic acid      | 1949219.0                    | 4597620.0                    |
| ## UDP-Hexose         | 6906863.5                    | 15428916.6                   |
| ## N2-Acetylornithine | 6976350.9                    | 9022753.7                    |
| ## Cytidine           | 12895117.9                   | 24530112.6                   |
| ##                    | X4106.Rotation.0.cGy.sGCR    | X4181.Rotation.0.cGy.sGCR    |
| ## 7-Methylguanine    | 695612.2                     | 879101.5                     |
| ## 8-oxoG             | 158235.4                     | 283070.3                     |
| ## Succinic acid      | 1754963.8                    | 3197623.6                    |
| ## UDP-Hexose         | 7556849.5                    | 7254077.0                    |
| ## N2-Acetylornithine | 2391176.0                    | 6318207.5                    |
| ## Cytidine           | 13764348.2                   | 15247356.0                   |
| ##                    | X4010.Rotation.50.cGy.sGCR   | X4015.Rotation.50.cGy.sGCR   |
| ## 7-Methylguanine    | 681938.9                     | 720233.5                     |
| ## 8-oxoG             | 180135.3                     | 126472.9                     |
| ## Succinic acid      | 2709254.6                    | 1372989.5                    |
| ## UDP-Hexose         | 6645396.3                    | 6568165.1                    |
| ## N2-Acetylornithine | 4007727.3                    | 2091646.4                    |
| ## Cytidine           | 16284408.1                   | 13050608.5                   |
| ##                    | X4026.Rotation.50.cGy.sGCR   | X4104.Rotation.50.cGy.sGCR   |
| ## 7-Methylguanine    | 539155.3                     | 999193.8                     |
| ## 8-oxoG             | 159515.0                     | 141648.9                     |
| ## Succinic acid      | 1202132.2                    | 1573581.7                    |
| ## UDP-Hexose         | 4316828.6                    | 4601417.6                    |
| ## N2-Acetylornithine | 3289138.6                    | 1863805.1                    |
| ## Cytidine           | 13636496.9                   | 19736061.3                   |
| ##                    | X4106.Rotation.50.cGy.sGCR   | X4181.Rotation.50.cGy.sGCR   |
| ## 7-Methylguanine    | 742604.5                     | 726137.5                     |
| ## 8-oxoG             | 168192.3                     | 362713.4                     |
| ## Succinic acid      | 1215232.2                    | 1397347.8                    |
| ## UDP-Hexose         | 4417412.1                    | 6251630.3                    |
| ## N2-Acetylornithine | 3310211.9                    | 748537.8                     |
| ## Cytidine           | 15942954.5                   | 18172420.4                   |
| ##                    | X4010.Rotation.100.cGy.Gamma | X4015.Rotation.100.cGy.Gamma |
| ## 7-Methylguanine    | 852673.3                     | 495796.2                     |
| ## 8-oxoG             | 181505.1                     | 141387.4                     |
| ## Succinic acid      | 1377384.1                    | 929656.9                     |
| ## UDP-Hexose         | 5674842.5                    | 3208101.4                    |
| ## N2-Acetylornithine | 2729940.0                    | 2484634.0                    |

```
## Cytidine 16808848.7 10714629.6
## X4026.Rotation.100.cGy.Gamma X4104.Rotation.100.cGy.Gamma
## 7-Methylguanine 412368.2 839635.4
## 8-oxoG 167216.6 145454.0
## Succinic acid 883612.0 1520365.0
## UDP-Hexose 3014394.0 5319247.5
## N2-Acetylornithine 3169495.7 1856190.8
## Cytidine 12388065.4 21641984.5
## X4106.Rotation.100.cGy.Gamma X4181.Rotation.100.cGy.Gamma
## 7-Methylguanine 968561.4 494151.3
## 8-oxoG 146946.2 193544.4
## Succinic acid 1377751.2 1099912.1
## UDP-Hexose 8858677.0 4412887.9
## N2-Acetylornithine 3491130.5 5306185.2
## Cytidine 19481050.0 10358174.2
## QC.4 QC.5 QC.6 QC.7 QC.8
## 7-Methylguanine 1060220.7 1049436.6 1077141.7 1013333.2 1016168.2
## 8-oxoG 185152.7 175179.9 177597.1 181499.5 180409.9
## Succinic acid 2815257.8 2934349.4 2792515.0 2924420.0 2841129.7
## UDP-Hexose 8753987.6 9020643.0 8468248.6 8765514.5 9100210.7
## N2-Acetylornithine 4664410.7 4860222.9 4737857.1 4921479.2 5052937.7
## Cytidine 19999577.5 19028666.4 19904394.3 18498189.8 19694245.4
## QC.End
## 7-Methylguanine 1050648.8
## 8-oxoG 187403.9
## Succinic acid 2756876.0
## UDP-Hexose 8853995.1
## N2-Acetylornithine 4844924.3
## Cytidine 19594528.0
```

With 36 biological samples plus the QC samples, the output is a little lengthy/crowded. When R does not have enough room to display a large object, the columns may wrap in the console or rendered PDF, making the output difficult to interpret. To keep things manageable, we will use square brackets and the `:` sequence operator to view only the first five rows and first five columns.

```
metab[1:5, 1:5]
```

```
## X4010.Dish.0.cGy.sGCR X4015.Dish.0.cGy.sGCR
## 7-Methylguanine 2769164.5 1368913.2
## 8-oxoG 246277.7 323311.2
## Succinic acid 6639808.8 1308332.8
## UDP-Hexose 8713523.5 3465812.5
## N2-Acetylornithine 3939274.4 5040621.4
## X4026.Dish.0.cGy.sGCR X4104.Dish.0.cGy.sGCR
## 7-Methylguanine 1065909.6 1165668.6
## 8-oxoG 320123.1 155842.3
## Succinic acid 3381226.0 4459418.9
## UDP-Hexose 7969206.4 7783445.5
## N2-Acetylornithine 6288896.4 2967679.9
## X4106.Dish.0.cGy.sGCR
## 7-Methylguanine 1039749.8
## 8-oxoG 243354.4
## Succinic acid 4671689.9
## UDP-Hexose 11324223.7
## N2-Acetylornithine 5928247.3
```

We can look at the first few column names of metab

```
head(names(metab))
```

```
## [1] "X4010.Dish.0.cGy.sGCR" "X4015.Dish.0.cGy.sGCR" "X4026.Dish.0.cGy.sGCR"
## [4] "X4104.Dish.0.cGy.sGCR" "X4106.Dish.0.cGy.sGCR" "X4181.Dish.0.cGy.sGCR"
```

as well as the row names

```
head(row.names(metab))
```

```
## [1] "7-Methylguanine"      "8-oxoG"                "Succinic acid"
## [4] "UDP-Hexose"           "N2-Acetylornithine"     "Cytidine"
```

What does our sample meta data look like?

```
head(sample_meta)
```

```
##      abundance_column      Group Type rad_dose_cGy rad_type      color
## 1 X4010.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
## 2 X4015.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
## 3 X4026.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
## 4 X4104.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
## 5 X4106.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
## 6 X4181.Dish.0.cGy.sGCR Dish 0 cGy sGCR Dish      0      sGCR firebrick
```

Let us use the `subset()` function on `sample_meta`, which takes a data frame as one argument and a column name, a logical operator, and the column value as another argument. This returns a filtered data frame that only has matching the column values. Note we can use other logical operators here like `!=` (not equals).

```
sample_meta_dish = subset(sample_meta, Type == "Dish")
```

We can now use a very useful trick that forms the basis for most of our upcoming analyses. Say we only wanted the columns from the metabolite abundance for Dish samples. We simply use `sample_meta_dish` to get the columns we want from the metabolite data frame:

```
metab[1:3, sample_meta_dish$abundance_column]
```

```
##      X4010.Dish.0.cGy.sGCR X4015.Dish.0.cGy.sGCR
## 7-Methylguanine          2769164.5          1368913.2
## 8-oxoG                   246277.7           323311.2
## Succinic acid           6639808.8          1308332.8
##      X4026.Dish.0.cGy.sGCR X4104.Dish.0.cGy.sGCR
## 7-Methylguanine          1065909.6          1165668.6
## 8-oxoG                   320123.1           155842.3
## Succinic acid           3381226.0          4459418.9
##      X4106.Dish.0.cGy.sGCR X4181.Dish.0.cGy.sGCR
## 7-Methylguanine          1039749.8          1013177.5
## 8-oxoG                   243354.4           198652.8
## Succinic acid           4671689.9          6479653.7
##      X4010.Dish.50.cGy.sGCR X4015.Dish.50.cGy.sGCR
## 7-Methylguanine          1926916.3          1119515.3
## 8-oxoG                   149197.2           205922.1
## Succinic acid           3160987.6          2836311.5
##      X4026.Dish.50.cGy.sGCR X4104.Dish.50.cGy.sGCR
## 7-Methylguanine          678706.2          1210638.2
## 8-oxoG                   150448.2           192363.2
## Succinic acid           2518031.3          1901065.0
##      X4106.Dish.50.cGy.sGCR X4181.Dish.50.cGy.sGCR
```

|                             |                          |           |
|-----------------------------|--------------------------|-----------|
| ## 7-Methylguanine          | 1008742.6                | 1154117.9 |
| ## 8-oxoG                   | 147589.9                 | 169534.6  |
| ## Succinic acid            | 2442808.7                | 3600233.3 |
| ## X4010.Dish.100.cGy.Gamma | X4015.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 1650095.3                | 1055828.9 |
| ## 8-oxoG                   | 210190.9                 | 179170.6  |
| ## Succinic acid            | 4005127.2                | 2054376.3 |
| ## X4026.Dish.100.cGy.Gamma | X4104.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 752840.0                 | 896385.5  |
| ## 8-oxoG                   | 149312.2                 | 158024.7  |
| ## Succinic acid            | 4012117.3                | 1723168.6 |
| ## X4106.Dish.100.cGy.Gamma | X4181.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 946172.7                 | 973979.7  |
| ## 8-oxoG                   | 176401.9                 | 200370.5  |
| ## Succinic acid            | 2077958.0                | 2353969.8 |

This code produces the same result:

```
metab[1:3, sample_meta_dish[, "abundance_column"]]
```

|                             |                          |           |
|-----------------------------|--------------------------|-----------|
| ## X4010.Dish.0.cGy.sGCR    | X4015.Dish.0.cGy.sGCR    |           |
| ## 7-Methylguanine          | 2769164.5                | 1368913.2 |
| ## 8-oxoG                   | 246277.7                 | 323311.2  |
| ## Succinic acid            | 6639808.8                | 1308332.8 |
| ## X4026.Dish.0.cGy.sGCR    | X4104.Dish.0.cGy.sGCR    |           |
| ## 7-Methylguanine          | 1065909.6                | 1165668.6 |
| ## 8-oxoG                   | 320123.1                 | 155842.3  |
| ## Succinic acid            | 3381226.0                | 4459418.9 |
| ## X4106.Dish.0.cGy.sGCR    | X4181.Dish.0.cGy.sGCR    |           |
| ## 7-Methylguanine          | 1039749.8                | 1013177.5 |
| ## 8-oxoG                   | 243354.4                 | 198652.8  |
| ## Succinic acid            | 4671689.9                | 6479653.7 |
| ## X4010.Dish.50.cGy.sGCR   | X4015.Dish.50.cGy.sGCR   |           |
| ## 7-Methylguanine          | 1926916.3                | 1119515.3 |
| ## 8-oxoG                   | 149197.2                 | 205922.1  |
| ## Succinic acid            | 3160987.6                | 2836311.5 |
| ## X4026.Dish.50.cGy.sGCR   | X4104.Dish.50.cGy.sGCR   |           |
| ## 7-Methylguanine          | 678706.2                 | 1210638.2 |
| ## 8-oxoG                   | 150448.2                 | 192363.2  |
| ## Succinic acid            | 2518031.3                | 1901065.0 |
| ## X4106.Dish.50.cGy.sGCR   | X4181.Dish.50.cGy.sGCR   |           |
| ## 7-Methylguanine          | 1008742.6                | 1154117.9 |
| ## 8-oxoG                   | 147589.9                 | 169534.6  |
| ## Succinic acid            | 2442808.7                | 3600233.3 |
| ## X4010.Dish.100.cGy.Gamma | X4015.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 1650095.3                | 1055828.9 |
| ## 8-oxoG                   | 210190.9                 | 179170.6  |
| ## Succinic acid            | 4005127.2                | 2054376.3 |
| ## X4026.Dish.100.cGy.Gamma | X4104.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 752840.0                 | 896385.5  |
| ## 8-oxoG                   | 149312.2                 | 158024.7  |
| ## Succinic acid            | 4012117.3                | 1723168.6 |
| ## X4106.Dish.100.cGy.Gamma | X4181.Dish.100.cGy.Gamma |           |
| ## 7-Methylguanine          | 946172.7                 | 973979.7  |
| ## 8-oxoG                   | 176401.9                 | 200370.5  |

```
## Succinic acid                2077958.0        2353969.8
```

This approach avoids hardcoding the samples where we specify the column numbers instead of the name of the samples:

```
metab[1:3, 1:18]
```

```
##          X4010.Dish.0.cGy.sGCR X4015.Dish.0.cGy.sGCR
## 7-Methylguanine          2769164.5          1368913.2
## 8-oxoG                   246277.7           323311.2
## Succinic acid            6639808.8          1308332.8
##          X4026.Dish.0.cGy.sGCR X4104.Dish.0.cGy.sGCR
## 7-Methylguanine          1065909.6          1165668.6
## 8-oxoG                   320123.1           155842.3
## Succinic acid            3381226.0          4459418.9
##          X4106.Dish.0.cGy.sGCR X4181.Dish.0.cGy.sGCR
## 7-Methylguanine          1039749.8          1013177.5
## 8-oxoG                   243354.4           198652.8
## Succinic acid            4671689.9          6479653.7
##          X4010.Dish.50.cGy.sGCR X4015.Dish.50.cGy.sGCR
## 7-Methylguanine          1926916.3          1119515.3
## 8-oxoG                   149197.2           205922.1
## Succinic acid            3160987.6          2836311.5
##          X4026.Dish.50.cGy.sGCR X4104.Dish.50.cGy.sGCR
## 7-Methylguanine          678706.2           1210638.2
## 8-oxoG                   150448.2           192363.2
## Succinic acid            2518031.3          1901065.0
##          X4106.Dish.50.cGy.sGCR X4181.Dish.50.cGy.sGCR
## 7-Methylguanine          1008742.6          1154117.9
## 8-oxoG                   147589.9           169534.6
## Succinic acid            2442808.7          3600233.3
##          X4010.Dish.100.cGy.Gamma X4015.Dish.100.cGy.Gamma
## 7-Methylguanine          1650095.3          1055828.9
## 8-oxoG                   210190.9           179170.6
## Succinic acid            4005127.2          2054376.3
##          X4026.Dish.100.cGy.Gamma X4104.Dish.100.cGy.Gamma
## 7-Methylguanine          752840.0           896385.5
## 8-oxoG                   149312.2           158024.7
## Succinic acid            4012117.3          1723168.6
##          X4106.Dish.100.cGy.Gamma X4181.Dish.100.cGy.Gamma
## 7-Methylguanine          946172.7           973979.7
## 8-oxoG                   176401.9           200370.5
## Succinic acid            2077958.0          2353969.8
```

What if columns 1-18 are originally Dish, but then we change the data by adding or removing samples? Columns 1-18 may no longer correspond to the Dish Type, and our subsequent analyses will be wrong. By using the `abundance_column` from the `sample_meta`, we can explicitly use Dish sample names to ensure we always get the data for Dish samples, regardless of their order. Importantly, this only works if `header = TRUE` was used when we read in the metabolite data such that `sample_meta_dish$abundance_column` corresponds to the header/column names of `metab`. If we need to delete a specific sample, then we should refer to it by its specific name (e.g. 4010.Dish.0.cGy.sGCR) instead of the specific column (e.g. column 14).

## Questions

For this and future questions, create your own chunk in this markdown file (Code -> Insert Chunk). You can also work in the console.



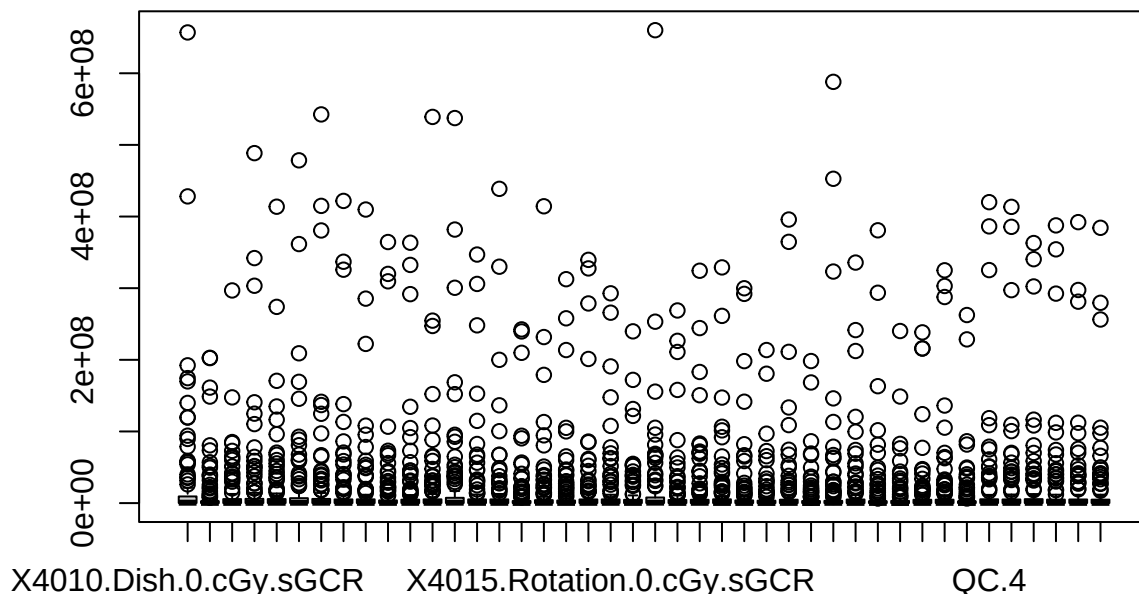
1. Data frames are examples of an object. Different objects have different properties. `metab` and `sample_meta` are both data frames. What happens if you put `nrow(metab)` in your chunk and run it? `ncol(metab)`? `dim(metab)`? `summary(metab)`?
2. Make a variable called `fav_num`, and assign your favorite number to it. What happens when you try to get the number of rows from it? What about `summary()`?
3. Try printing the first few row names using `head(row.names(sample_meta))`. Do any of them match the column names of `metab`?
4. What does `head(sample_meta, 5)` do? `head(sample_meta, 10)`? What does the second argument of `head` control?
5. Create your own subset of `sample_meta` for only the Rotation Type. Can you use it to select the corresponding columns from `metab`?
6. Get the average of the the column `X4010.Dish.0.cGy.sGCR` across all metabolites while ignoring missing values with `mean(metab[, "X4010.Dish.0.cGy.sGCR"], na.rm = TRUE)`. How would find the mean of Succinic acid across all samples? Hint: based on how R handles rows, you will need to nest functions to make sure R treats the data as numeric values that it can take the mean of and not as a one row data frame. The command will look something like `mean(as.numeric(), na.rm = TRUE)`.
7. Find the mean of your favorite amino acid in only the Rotation samples.

## Visualize data

### Boxplot

R has a built-in `boxplot()` function that can take a data frame as input:

```
boxplot(metab[, sample_meta$abundance_column])
```



The result is not very informative. metabolomics data, like most mass spectrometry data, tends to follow an exponential distribution. This means many values cluster at the lower end, while a few extreme values stretch the plot upward, and plotting the values spans many orders of magnitude. As a result, it can be difficult to see detail in the lower part of the distribution. Additionally, the sample labels are not displaying clearly, making the plot harder to interpret.

To improve this, we will start by applying a log2 transformation to the intensity values. This transformation makes the distribution more symmetric and closer to log-normal, which helps with visualization and also allows us to use parametric statistical tests like the t-test to compare group means.

```
metab[, sample_meta$abundance_column] = log2(metab[, sample_meta$abundance_column])
metab[1:5, 1:5]
```

```
##              X4010.Dish.0.cGy.sGCR X4015.Dish.0.cGy.sGCR
## 7-Methylguanine          21.40102          20.38460
## 8-oxoG                   17.90993          18.30256
## Succinic acid            22.66271          20.31930
## UDP-Hexose               23.05482          21.72476
## N2-Acetylornithine       21.90950          22.26517
##              X4026.Dish.0.cGy.sGCR X4104.Dish.0.cGy.sGCR
## 7-Methylguanine          20.02365          20.15273
## 8-oxoG                   18.28827          17.24973
## Succinic acid            21.68912          22.08842
## UDP-Hexose               22.92600          22.89198
## N2-Acetylornithine       22.58438          21.50090
##              X4106.Dish.0.cGy.sGCR
## 7-Methylguanine          19.98781
## 8-oxoG                   17.89270
## Succinic acid            22.15551
## UDP-Hexose               23.43291
## N2-Acetylornithine       22.49917
```

Now that the data has been log2-transformed, let us replot it. Note that due to size constraints, there may be some overlap in the plot elements when this is knitted into a PDF.

```
# Increase bottom margin to fit longer x-axis labels; the
# pattern is bottom, left, top, right
par(mar = c(10, 4, 2, 1))

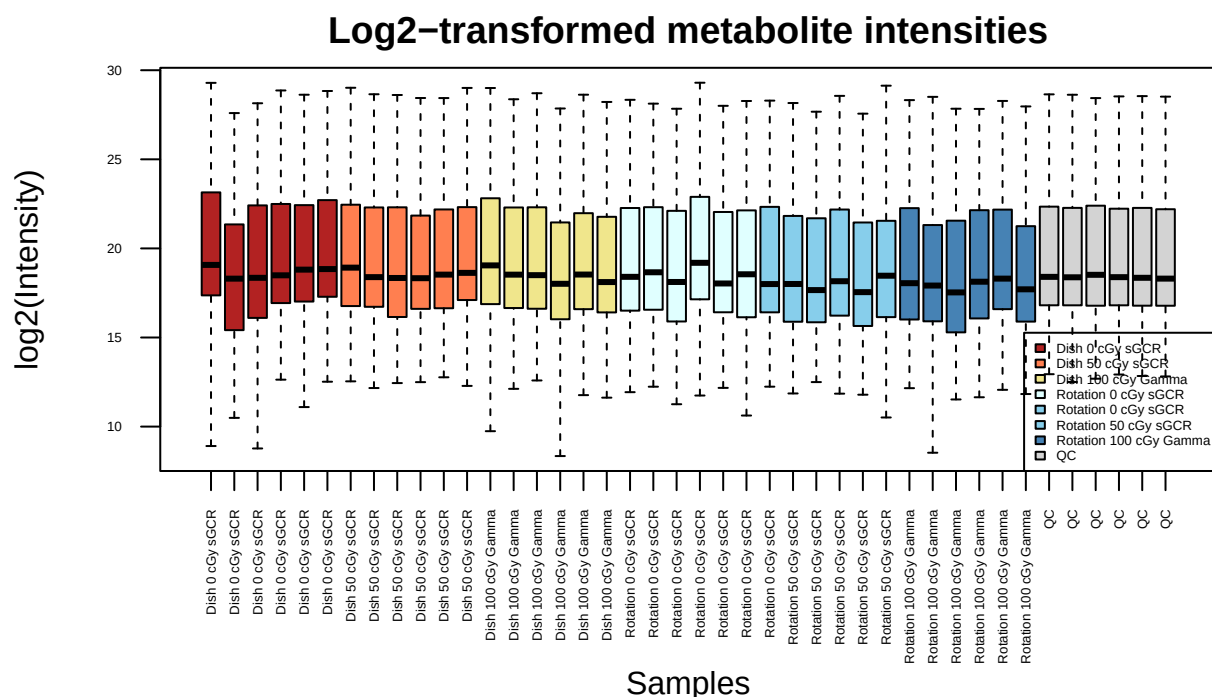
# Draw boxplot; # use color directly from sample_metadata;
# x-axis labels = Group names

boxplot(metab[, sample_meta$abundance_column], col = sample_meta$color,
        names = sample_meta$Group, las = 2, xlab = "", ylab = "log2(Intensity)",
        main = "Log2-transformed metabolite intensities", cex.axis = 0.4)

title(xlab = "Samples", line = 5)

# Add legend (one entry per unique Group-color combo)
legend_Groups = unique(sample_meta[, c("Group", "color")])

# cex allows you to scale the legend
legend("bottomright", legend = legend_Groups$Group, fill = legend_Groups$color,
       cex = 0.4)
```



It is now much easier to see what is happening in the data. After applying the log2 transformation, the intensities no longer span several orders of magnitude. Instead, the values are compressed into a more manageable range, roughly between 5 and 30 on the log2 scale. Notice how the medians nearly form a straight line across the boxes. This is a sign that the data has been normalized or otherwise processed, in this case using SERRF (<https://slfan2013.github.io/SERRF-online/>).

We should always watch for samples that behave abnormally in plots like these. For example, a sample with a much lower median than the others, even after normalization, might indicate a technical issue or sample failure. Color is also a helpful tool in data visualization. Here, the use of distinct colors makes it easy to separate the groups visually, even without referring to the legend.

## Boxplot with ggplot2

While we will emphasize a lot of plotting using base R in this workshop, we will also take a look at the ggplot2 package. ggplot2 is part of the tidyverse suite of R packages, and it's a powerful system for creating more polished and customizable plots. However, it can require more code and additional data preparation, especially for boxplots across multiple samples. When possible, we will stick to simpler base R plots to keep things clear, and introduce ggplot2 when we want more flexibility or aesthetics, but for the sake of demonstration let us make a box plot with ggplot2.

First this requires some additional packages:

```
install.packages(c("tidyr", "dplyr", "ggplot2"))
```

```
library(tidyr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
```

```

##
##      intersect, setdiff, setequal, union
library(ggplot2)

# Subset metabolite abundance data
expression_data = metab[, sample_meta$abundance_column, drop = FALSE]

# Preserve metabolite names before reshaping
expression_data$metabolite = rownames(expression_data)

# Reshape to long format and add sample metadata
long_df = expression_data %>%
  pivot_longer(cols = -metabolite, names_to = "abundance_column",
    values_to = "intensity") %>%
  left_join(sample_meta[, c("abundance_column", "Group", "color")],
    by = "abundance_column") %>%
  filter(!is.na(intensity), intensity > 0)

# Order samples exactly as they appear in sample_meta
long_df$abundance_column = factor(long_df$abundance_column, levels = sample_meta$abundance_column)

# Order groups by their first appearance in sample_meta
group_order = unique(sample_meta$Group)

long_df$Group = factor(long_df$Group, levels = group_order)

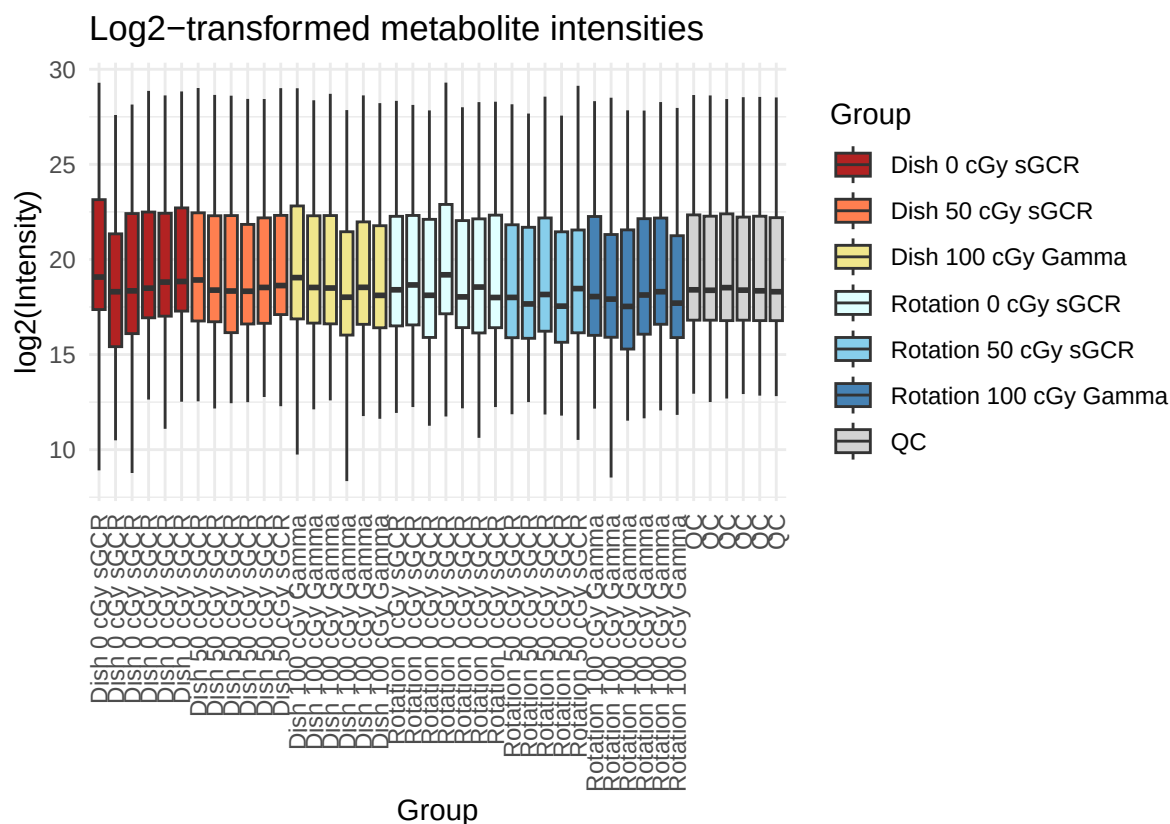
# Match each sample column to its displayed group label
x_labels = setNames(sample_meta$Group, sample_meta$abundance_column)

# Create a named group-to-color vector in the same order
group_colors = sample_meta %>%
  distinct(Group, color) %>%
  mutate(Group = factor(Group, levels = group_order)) %>%
  arrange(Group)

group_colors = setNames(group_colors$color, as.character(group_colors$Group))

# Plot one box per sample
ggplot(long_df, aes(x = abundance_column, y = intensity, fill = Group)) +
  geom_boxplot(outlier.shape = 21, outlier.size = 1) + scale_x_discrete(labels = x_labels,
    drop = FALSE) + scale_fill_manual(values = group_colors,
    breaks = group_order, drop = FALSE) + theme_minimal() + theme(axis.text.x = element_text(angle = 90,
    vjust = 0.5, hjust = 1), plot.margin = margin(10, 10, 10,
    20)) + labs(x = "Group", y = "log2(Intensity)", title = "Log2-transformed metabolite intensities",
    fill = "Group")

```



## Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a very important dimension reduction technique. PCA works by identifying new axes, called principal components (PCs), that capture the most variability in the data. Instead of plotting hundreds or thousands of features, PCA reduces this to just a few new dimensions that explain the overall structure. We usually focus on the first two principal components, since they explain the largest amount of variation, but additional components can also be explored. Since PCA cannot handle missing values, we need to check if we have missing values. If we do have any, then we would need to fill them in, or “impute” them.

```
table(is.na(metab))
```

```
##
## FALSE
## 4998
```

We only see FALSE when checking if the values in metab are missing, so we have a complete matrix with no missing values. We can now run PCA using the `prcomp()` function. This function expects samples to be rows and variables to be columns, so we need to transpose the data first. We also set `center = TRUE` and `scale = TRUE` to standardize the data. Scaling and centering (also known as z-scoring) is important because it puts all features on the same scale. Without it, features with larger values or more variability, like a particularly abundant or noisy metabolite, can dominate the analysis. This would make the PCA reflect the behavior of just a few features rather than the overall data structure. Standardizing ensures that each feature contributes equally to the principal components.

```
# Transpose the data so that rows are samples and columns
# are features
metab_for_pca = t(metab)
```

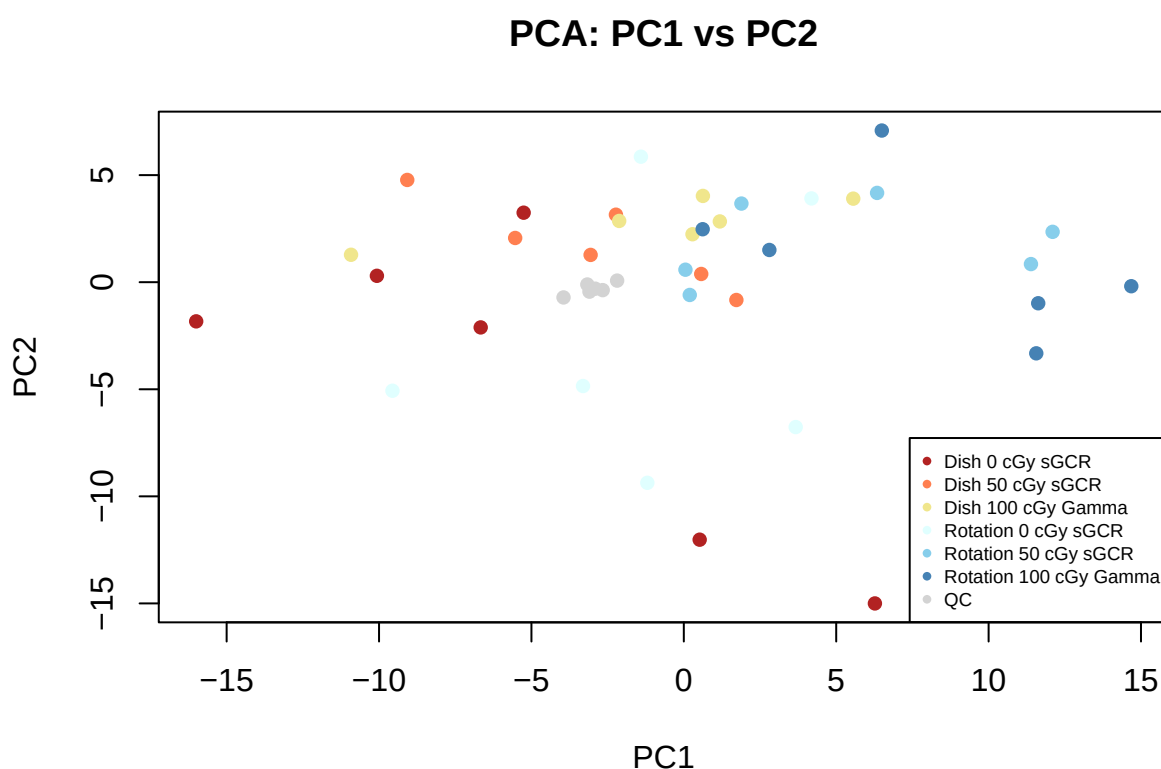
```
# Run PCA
pca_result = prcomp(metab_for_pca, center = TRUE, scale = TRUE)

Once PCA is complete, we will visualize the first two principal components in a scatter plot, colored by group
using the values from sample_meta$color.

# Extract PCA scores
pca_scores = pca_result$x

# Plot the first two principal components using base R
plot(pca_scores[, 1], pca_scores[, 2], col = sample_meta$color,
     pch = 16, xlab = "PC1", ylab = "PC2", main = "PCA: PC1 vs PC2")

# Add a legend to explain the colors
legend("bottomright", legend = unique(sample_meta$Group), col = unique(sample_meta$color),
     pch = 16, cex = 0.6)
```



We can see some separation in spread across PC1 and PC2. This suggests there are some differences between groups, but we will need to perform differential abundance analysis to be sure. The QC samples are in very close proximity to each other, which is a very good sign so we do not have data quality concerns.

PCA is also a helpful tool for spotting technical artifacts. If we saw unexpected clustering by something unrelated to biology, such as sample order, batch, or run date, that could indicate the presence of batch effects or technical bias. Likewise, if a sample falls far from all the others, it might be a true outlier due to poor quality or a possible sample swap. In this case, we do not see any signs of those issues.

## Heatmap

Heatmaps are another useful way to visualize our data while performing hierarchical clustering. A heatmap shows the intensity of many features, such as metabolites, across all samples, using color to represent relative abundance. Clustering is applied to both rows (features) and columns (samples) to group together those with

similar abundance patterns. Note that clustering might not work if too many values are missing, but we do not have to worry about that here. If we did, then we would use imputation or missing value filtering.

We typically do not include all metabolites in the heatmap. In any omics experiment, many features will not vary much across samples. Including these adds noise and makes patterns harder to see. Instead, we focus on a subset of the most variable features. One common approach is to use standard deviation to identify the most variable metabolites. Here, we will select the top 25 metabolites based on their standard deviation.

To do this, we use the `apply()` function, which applies a function to either the rows or columns of a data frame. By setting `MARGIN = 1`, we tell R to apply the `sd()` (standard deviation) function to each row (i.e., each metabolite).

```
# calculate the standard deviation across all rows for
# every metabolite
metabolite_sds = apply(metab, MARGIN = 1, sd)
# sort the data so that the metabolites with the top
# standard deviation are in the first rows
top_metabolites = names(sort(metabolite_sds, decreasing = TRUE))[1:25]
# make a new data frame with only the top metabolites for
# the heatmap
top_metab = metab[top_metabolites, ]
```

Before plotting, we need to center and scale the data by performing a z-score transformation on each row. This step is important. Just like in PCA, large differences in raw intensity values can overwhelm the clustering and lead to misleading patterns. Without standardization, metabolites with higher average abundance will cluster together not because they behave similarly across samples, but simply because their overall values are larger. `scale()` scales columns by default. Since our rows are metabolites and columns are samples, we need to transpose with the `t()` function so each metabolite temporarily becomes a column, scale it across samples, then transpose back.

```
# z-score normalization (row-wise)
z_top_metab = t(scale(t(top_metab)))
```

After scaling, the heatmap will better reflect biological patterns of change rather than differences in measurement scale. Clusters of samples and features may emerge that reflect similarities between groups. Similar to the PCA, the heatmap can also help detect/visualize outliers.

To create our heatmap, we will use the `ComplexHeatmap` package. This package is part of Bioconductor (<https://bioconductor.org>), which is a specialized repository for bioinformatics tools in R. Unlike CRAN, which hosts general-purpose R packages, Bioconductor focuses on packages for analyzing genomic and other biological data. You install Bioconductor packages using the `BiocManager` package.

You can do so with the following code. This block checks if `BiocManager` is installed, installs it if needed, and then uses it to install `ComplexHeatmap` (<https://www.bioconductor.org/packages/release/bioc/html/ComplexHeatmap.html>). You can copy and paste this into your R console:

```
if (!require("BiocManager", quietly = TRUE)) install.packages("BiocManager")
BiocManager::install("ComplexHeatmap")
```

Once the package is installed, we can prepare our data. We will select the top 25 most variable metabolites based on standard deviation, apply z-score scaling to center and standardize each row (metabolite), and color the columns (samples) by Type/Dose from `sample_meta`.

```
# Load ComplexHeatmap
library(ComplexHeatmap)
```

```
## Loading required package: grid
## =====
```

```

## ComplexHeatmap version 2.24.0
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##   genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## =====
# Define annotation colors
type_colors = c(Dish = "#FDB462", Rotation = "#B3DE69", QC = "lightgrey")

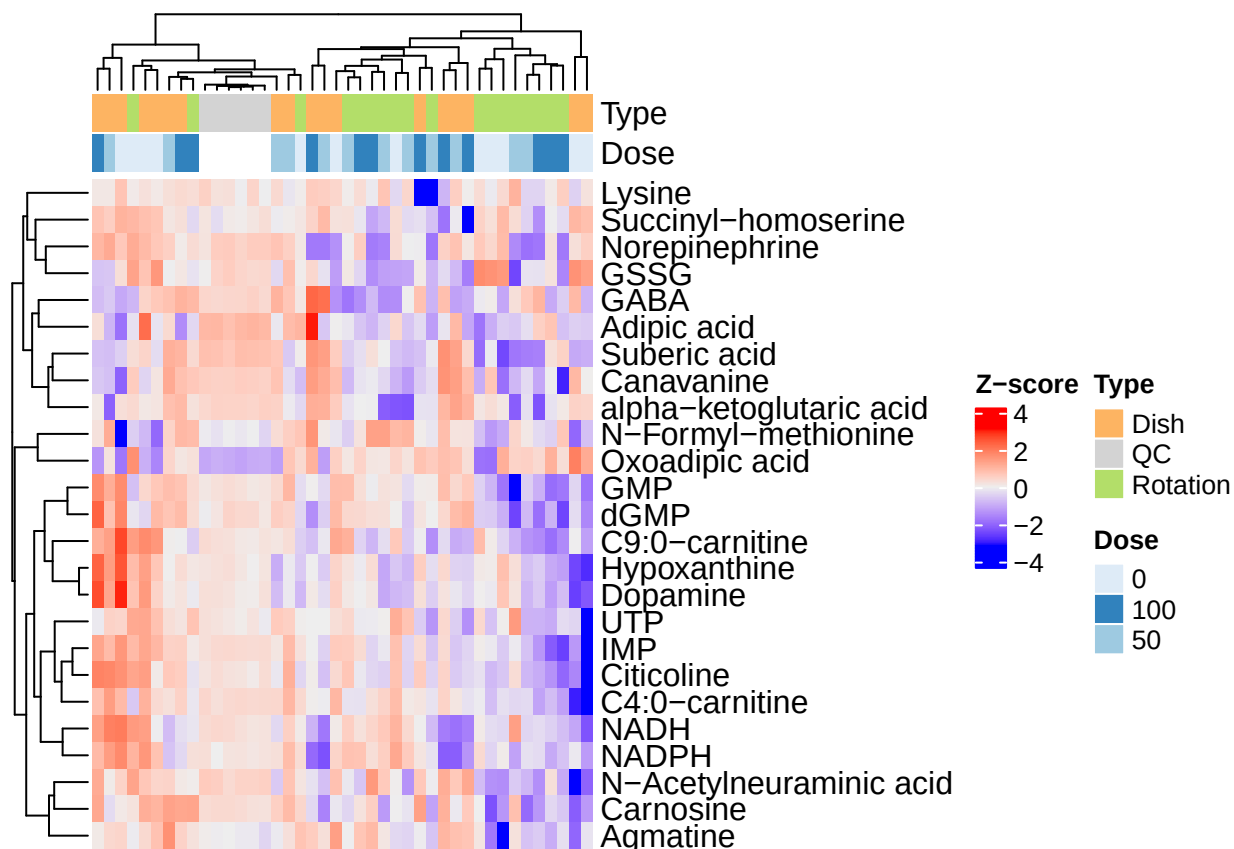
# Light to dark gradient with increasing radiation dose
dose_colors = c(`0` = "#DEEBF7", `50` = "#9ECAE1", `100` = "#3182BD")

# Create column annotations
column_annot = HeatmapAnnotation(Type = sample_meta$Type, Dose = as.character(sample_meta$rad_dose_cGy),
  col = list(Type = type_colors, Dose = dose_colors), na_col = "white")

# Draw heatmap
Heatmap(z_top_metab, name = "Z-score", top_annotation = column_annot,
  show_column_names = FALSE, show_row_names = TRUE, cluster_rows = TRUE,
  cluster_columns = TRUE)

```





## Questions

1. The dendrogram at the top of the heatmap represents the similarity among samples. Do the QC samples cluster more closely with one another or with the biological samples?
2. Try setting `cluster_rows = FALSE` and `cluster_columns = FALSE` in your heatmap to see how the visualization changes. This could be useful if you wanted particular samples/metabolites in a specific order.
3. Create a new data frame with just the top 10 most variable metabolites (by standard deviation). Try plotting a heatmap using just this smaller set.
4. What happens if you make a heatmap with `top_metab` instead of `z_top_metab`?

## Differential abundance

### ID mapping

As part of the differential abundance analysis, we will test if the metabolites that differ between groups are enriched in specific biological pathways more often than expected by chance. To perform this analysis, metabolite names must first be linked to identifiers recognized by a pathway database such as KEGG.

We will use a mapping table generated previously with MetaboAnalyst. Because metabolite names are not fully standardized, automated mapping often requires manual review. A completed mapping table is provided so we can focus on the downstream analysis during the workshop.

```
# read in the ID mapping
id_mapping = read.csv("data/name_map_metaboanalyst_downloaded_2026-07-29.csv",
  header = TRUE, stringsAsFactors = FALSE)
```

```
# use the merge() function to combine two data frames;
# all.x = TRUE ensures that we do not drop rows if there is
# not a match between the two data frames we are merging
metab = merge(metab, id_mapping, by.x = "row.names", by.y = "Query",
              all.x = TRUE)
```

Since we used the row names to merge, there is a quirk in the behavior of merge where the row names of metab are numbers, and there is a column called Row.names that has our old row name values. To fix the row names of metab, simply run

```
# set the row names of metab
row.names(metab) = metab$Row.names
# delete the extra column to avoid confusion between
# Row.names and row.names
metab$Row.names = NULL
```

Let's take a look at metab by clicking on it in the environment or typing View(metab) in the console...

It worked but all of our metabolite annotation is in the last few columns. We want it in the first few columns so it is easier to see what metabolite we are dealing with. Let's rearrange the names of metab by reassigning metab based on the ordering of the columns we want to see:

```
metab = metab[, c("Match", "HMDB", "PubChem", "ChEBI", "KEGG",
                  "METLIN", "SMILES", "Comment", sample_meta$abundance_column)]
```

Look at metab again to verify it worked.

## Hypothesis testing

Now that we are happy with how the data looks and our metabolites are labeled with corresponding KEGG identifiers, we can ask which metabolites are differentially abundant between experimental conditions. A common way to test for differences in expression is to use a two-sample t-test. This test compares the means of two groups and asks if the observed difference could have occurred by chance.

The t-test returns a p-value, which tells us the probability of observing a difference as large (or larger) than what we saw if there were actually no real difference. A small p-value (typically less than 0.05) suggests that the observed difference is unlikely to be due to random chance.

However, statistical significance does not always mean biological significance. For example, a diabetes drug might reduce blood sugar by only 1 percent, say from 200 to 198. Such a change might be statistically significant in a large study but would not be meaningful for the patient. That is why we often pair p-values with effect size. In this case, we use log2 fold-change. A larger absolute fold-change is more likely to be biologically important. We will use a 1.5-fold change (or  $\log_2(1.5) = 0.5849625$ ).

We will compare two experimental groups and identify metabolites that show both statistical evidence of a difference and a meaningful change in abundance. There are several steps, so first we will again use our trick with the sample\_meta data to identify the sample columns for each group. It is a good idea to add the results to additional columns at the end of the dataset because it is easy in R and then we have both the abundance and the analysis results in the same place. Note that we make the choice not to hard code our variable names but instead call them group1 and group2. This means later as an exercise we can compare other groups by changing the below two lines instead of having to copy and paste/find and replace.

```
group1_samples = subset(sample_meta, Group == "Dish 100 cGy Gamma")$abundance_column
group2_samples = subset(sample_meta, Group == "Rotation 100 cGy Gamma")$abundance_column
```

We will again use apply, this time to calculate means. The results get assigned to new columns at the end of the metab data frame:

```
metab$group1_mean = apply(metab[, group1_samples], 1, function(x) mean(x,
  na.rm = TRUE))
metab$group2_mean = apply(metab[, group2_samples], 1, function(x) mean(x,
  na.rm = TRUE))
```

We will next calculate the log2 fold change (or the “log2 ratios”) between samples, but first a test from your college algebra days:  $\log_2(a/b) = ???$ .

Since our data is already log2 transformed, we can simply subtract the group means to compute the log2 fold-change.

```
# this subtracts across all rows
metab$group2_group1_log2fc = metab$group2_mean - metab$group1_mean
```

Next, we perform t-tests for every metabolite to compare abundance between group1 and group2.

```
# Perform t-tests row-wise
metab$pval = apply(metab, 1, function(x) {
  t.test(as.numeric(x[group1_samples]), as.numeric(x[group2_samples]))$p.value
})

# If you ever try to reuse this code but have missing
# values, then use this code instead:

# metab$pval = apply(metab, 1, function(x) { tryCatch({
# t.test( as.numeric(x[group1_samples]),
# as.numeric(x[group2_samples])) $p.value }, error =
# function(e) NA) })

# This tells R to try running the code, but if it fails
# (for example, due to too many NAs), it should return NA
# instead of crashing. This allows the loop to continue for
# the rest of the metabolites.
```

After calculating a p-value for each metabolite, we need to account for performing many statistical tests simultaneously. Without correction, some metabolites would appear significant by chance alone. We therefore adjust the p-values using the Benjamini–Hochberg method to control the false discovery rate. Because the differences between groups are modest, we will use an exploratory adjusted p-value threshold of 0.15. Among the metabolites called significant, the expected proportion of false discoveries is therefore controlled at approximately 15%. These results should therefore be considered hypothesis-generating and would require further validation.

```
# Adjust p-values using the Benjamini-Hochberg (FDR) method
metab$padj = p.adjust(metab$pval, method = "BH")
```

Now let us identify the metabolites that meet our criteria for differential abundance. We can do this in a single line using `abs()` to calculate the absolute value of the log2 fold change. This allows us to capture changes in either direction: metabolites that are at least 1.5-fold higher or at least 1.5-fold lower between groups. The `&` operator requires both the fold-change threshold and the adjusted p-value threshold to be met.

```
metab$differentially_expressed = abs(metab$group2_group1_log2fc) >=
  log2(1.5) & metab$padj <= 0.15
```

## Volcano plot

A volcano plot is a useful visualization tool for differential abundance analysis. It helps us quickly identify features that are statistically significant with large changes in abundance. The x-axis of the volcano plot

shows the log2 fold change, which represents how much a metabolite's expression increases or decreases. The y-axis shows  $-\log_{10}$  of the adjusted p-value (padj), so that more statistically significant metabolites appear higher up on the plot.

The reason it's called a "volcano" is because when most metabolites have small fold changes and only a few have large changes, the points tend to form a shape that resembles an erupting volcano. This shape can help us diagnose if our differential abundance comparison is working properly. If everything is clustered around zero or there are no significant values, it may indicate problems like low statistical power or technical noise.

To make a volcano plot, we need several pieces of information for each metabolite:

- the log2 fold change
- the adjusted p-value (padj)
- the  $-\log_{10}(\text{padj})$  for plotting on the y-axis
- the metabolite name or label
- the color (based on significance and direction of change)

Rather than calculating and tracking each of these separately, it's cleaner and easier to put everything into a single data frame. This allows us to work with the data in a structured way, label the most extreme values, and use that data frame directly in our plot.

```
volcano_df = data.frame(log2fc = metab$group2_group1_log2fc,  
  padj = metab$padj, name = row.names(metab))
```

Calculate  $-\log_{10}(\text{padj})$

```
volcano_df$neglog10padj = -log10(volcano_df$padj)
```

Assign colors by significance and direction

```
volcano_df$color = "gray"  
volcano_df$color[volcano_df$padj <= 0.15 & volcano_df$log2fc >  
  log2(1.5)] = "red"  
volcano_df$color[volcano_df$padj <= 0.15 & volcano_df$log2fc <  
  -log2(1.5)] = "blue"
```

Identify the most differentially abundant metabolites for labeling so we can display them in the plot

```
# Filter for significant metabolites only  
sig_df = volcano_df[volcano_df$padj <= 0.15, ]  
  
# Find top 5 significant upregulated (most positive log2fc)  
top5_up_idx = order(sig_df$log2fc, decreasing = TRUE)[1:5]  
  
# Find top 5 significant downregulated (most negative  
# log2fc)  
top5_down_idx = order(sig_df$log2fc)[1:5]  
  
# Get the original row numbers for labeling; this will be  
# used for text() momentarily  
label_idx = match(sig_df$name[c(top5_up_idx, top5_down_idx)],  
  volcano_df$name)
```

Create the volcano plot using base R

```
# Plot the volcano  
plot(volcano_df$log2fc, volcano_df$neglog10padj, col = adjustcolor(volcano_df$color,  
  alpha.f = 0.8), pch = 16, xlab = "Log2 Fold Change", ylab = "-log10 Adjusted p-value",  
  xlim = c(-3.5, 3.5), ylim = c(0, 1.5), main = "Volcano Plot")
```



```

# Create volcano_df_gg
volcano_df_gg = data.frame(log2fc = metab$group2_group1_log2fc,
  padj = metab$padj, name = row.names(metab))

# Calculate -log10 adjusted p-value
volcano_df_gg$neglog10padj = -log10(volcano_df_gg$padj)

# Assign Group labels for coloring
volcano_df_gg$Group = "Not significant"
volcano_df_gg$Group[volcano_df_gg$padj <= 0.15 & volcano_df_gg$log2fc >=
  log2(1.5)] = "Higher in group2"
volcano_df_gg$Group[volcano_df_gg$padj <= 0.15 & volcano_df_gg$log2fc <=
  -log2(1.5)] = "Higher in group1"

# Set factor levels for consistent legend order
volcano_df_gg$Group = factor(volcano_df_gg$Group, levels = c("Higher in group2",
  "Higher in group1", "Not significant"))

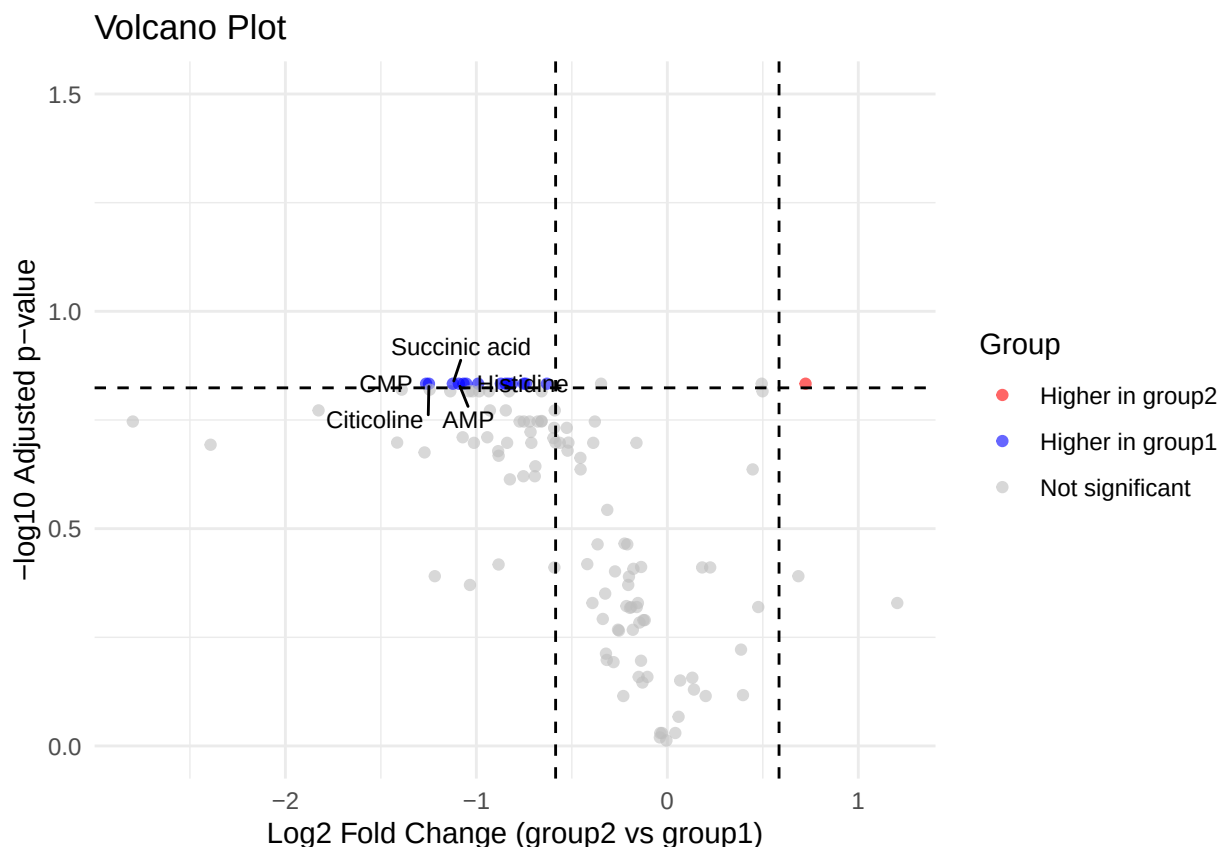
# Filter for complete, plottable data
volcano_df_gg_clean = subset(volcano_df_gg, is.finite(log2fc) &
  is.finite(neglog10padj))

# Identify top 5 down by fold change
down_df = subset(volcano_df_gg_clean, padj <= 0.15 & log2fc <=
  -log2(1.5))

label_df = head(down_df[order(down_df$log2fc), ], 5)

# Make the volcano plot
ggplot(volcano_df_gg_clean, aes(x = log2fc, y = neglog10padj,
  color = Group)) + geom_point(alpha = 0.6) + geom_vline(xintercept = c(-log2(1.5),
  log2(1.5)), linetype = "dashed") + geom_hline(yintercept = -log10(0.15),
  linetype = "dashed") + geom_text_repel(data = label_df, aes(label = name),
  color = "black", size = 3, max.overlaps = Inf, show.legend = FALSE) +
  scale_color_manual(values = c(`Higher in group2` = "red",
    `Higher in group1` = "blue", `Not significant` = "gray")) +
  coord_cartesian(ylim = c(0, 1.5)) + labs(title = "Volcano Plot",
  x = "Log2 Fold Change (group2 vs group1)", y = "-log10 Adjusted p-value",
  color = "Group") + theme_minimal()

```



## Pathway enrichment analysis with FELLA

Looking at differentially abundant metabolites one at a time can make it difficult to see the larger biological patterns in the data. Pathway enrichment analysis helps by grouping metabolites into known metabolic pathways and testing if any pathways contain more of our hits than expected by chance. This provides a higher-level view of the metabolic processes that may be altered between groups.

We will use the FELLA package to perform pathway enrichment using KEGG compound identifiers.

```
if (!require("BiocManager", quietly = TRUE)) install.packages("BiocManager")
BiocManager::install("FELLA")
```

FELLA compares the metabolites identified as differentially abundant with the metabolic pathways in its KEGG database and calculates a p-value for each pathway. We will then adjust these p-values for multiple testing using the Benjamini-Hochberg method.

For the sake of time, a prebuilt FELLA database is provided (if you are interested in generating your own say for a different organism, see `fella.R` in the `scripts/` folder). We first load this database, then extract the KEGG identifiers for metabolites that meet our differential-abundance criteria. Missing identifiers are removed, and `unique()` ensures that each KEGG compound is included only once. Finally, we use FELLA's hypergeometric method to test which pathways contain more of these metabolites than expected by chance.

```
library(FELLA)

fella_data = loadKEGGdata(databaseDir = "data/fella_hsa_hypergeom",
  internalDir = FALSE, loadMatrix = "hypergeom")

# Get KEGG IDs for differentially abundant metabolites
hits = unique(na.omit(subset(metab, differentially_expressed ==
```

```
TRUE)$KEGG))

# Run pathway enrichment
fella_results = enrich(compounds = hits, methods = "hypergeom",
  data = fella_data)
```

FELLA prints several status messages while loading the database. It may also note that diffusion and PageRank files are unavailable because the provided database contains only the components needed for hypergeometric enrichment. These messages can be ignored for this analysis.

```
# Create the FELLA pathway results table threshold = 1
# keeps all pathways so we can adjust the p-values
# ourselves
pathway_results = generateResultsTable(object = fella_results,
  data = fella_data, method = "hypergeom", threshold = 1)
```

```
## Writing hypergeom results...
```

```
## Done.
```

```
# Adjust pathway p-values for multiple testing
pathway_results$padj = p.adjust(pathway_results$p.value, method = "BH")
```

We can take the resulting pathway enrichment table and sort by adjusted p-value, and then plot the top pathways as a horizontal bar plot. The bars show the  $-\log_{10}$  adjusted p-value, so longer bars represent stronger evidence of pathway enrichment. The dashed vertical line marks an adjusted p-value of 0.05.

```
# Sort pathways from smallest to largest adjusted p-value
pathway_results = pathway_results[order(pathway_results$padj),
  ]

# Select the top n pathways
top_n = 10

top_kegg = head(pathway_results, top_n)

# Calculate -log10 adjusted p-value
top_kegg$neglog10padj = -log10(top_kegg$padj)

# Remove the organism description from the pathway names
top_kegg$short_name = sub(" - Homo sapiens \\(human\\)$", "",
  top_kegg$KEGG.name)

# Shorten pathway names longer than 40 characters
short_labels = ifelse(nchar(top_kegg$short_name) > 40, paste0(substr(top_kegg$short_name,
  1, 37), "..."), top_kegg$short_name)

# Create a light-to-dark blue color gradient
bar_colors = colorRampPalette(c("darkblue", "lightblue"))(nrow(top_kegg))

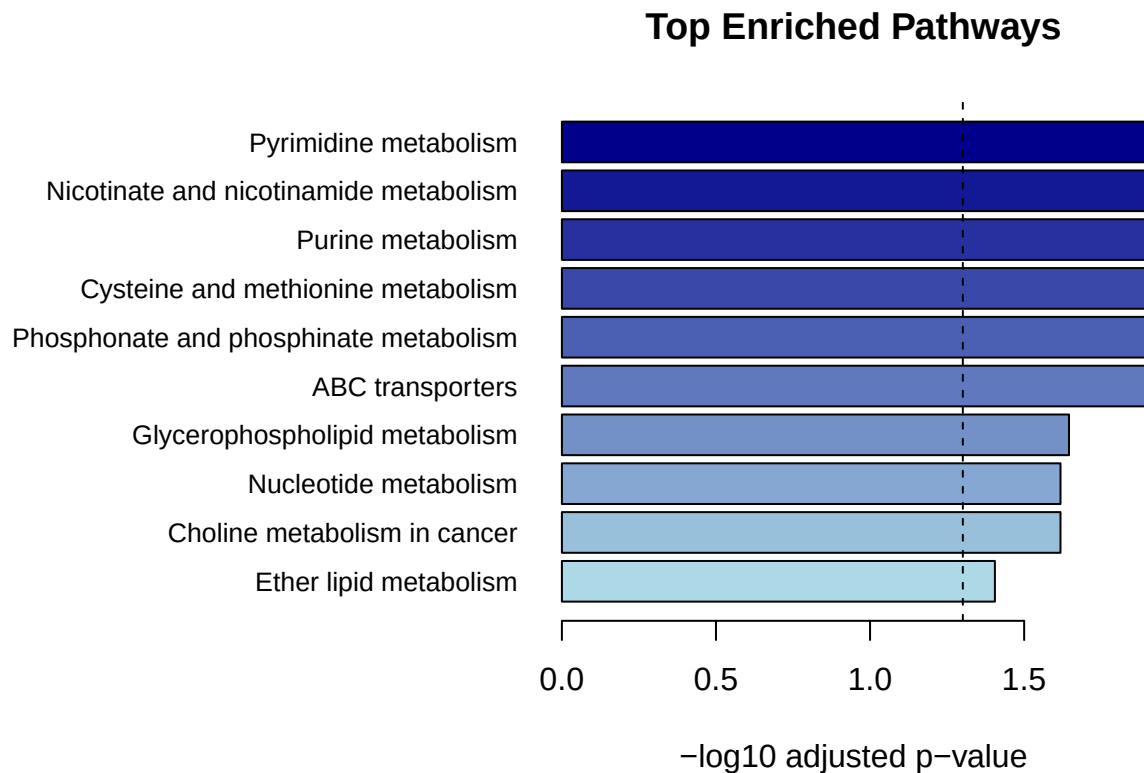
# Increase the left margin for pathway names
par(mar = c(5, 15, 4, 2))

# Plot the top 10 enriched pathways
barplot(rev(top_kegg$neglog10padj), names.arg = rev(short_labels),
  horiz = TRUE, col = rev(bar_colors), las = 1, xlab = "-log10 adjusted p-value",
```



```
main = "Top Enriched Pathways", cex.names = 0.8)

# Add a vertical line at adjusted p-value = 0.05
abline(v = -log10(0.05), lty = 2, col = "black")
```



## Save the results

Save the updated metab with metabolite annotations and differential abundance results:

```
write.table(metab, "work/metab_differential_abnd_2026-07-29.txt",
  sep = "\t", quote = F, row.names = T, col.names = NA)
```

Similarly for the pathway enrichment:

```
write.table(pathway_results, "work/pathway_enrichment_2026-07-29.txt",
  sep = "\t", quote = FALSE, row.names = FALSE)
```

## Questions

1. How many metabolites pass both the adjusted p-value and fold-change cutoffs?
2. What happens if you change the  $\log_2$  fold-change cutoff from  $\log_2(1.5)$  to  $\log_2(1.25)$  for differential abundance? Does this affect the number of differentially expressed metabolites?
3. Try making a histogram of all  $\log_2$  fold-changes using `hist()`. Does the distribution look symmetric?
4. Among metabolites with higher abundance in group 2, which has the 11th-smallest adjusted p-value?
5. Perform pathway enrichment analysis on the significantly downregulated metabolites, and make a barplot to visualize the results.

6. The choice of background is important in pathway enrichment analysis. If no background is supplied, FELLA uses all compounds represented in its KEGG-derived database as the comparison set. This may be inappropriate for targeted or semi-targeted experiments, because the measured metabolites are not a random sample of the entire metabolome. For example, an assay enriched for amino acids or nucleotides may make those pathways appear enriched simply because they are already overrepresented in the panel. Using all observed and successfully mapped metabolites as the background asks a more appropriate question: are the differentially abundant metabolites concentrated in a pathway relative to the metabolites that could actually have been identified as hits? Define the background as all metabolites observed in the experiment: `background = unique(na.omit(metab$KEGG))`. Then rerun `enrich()` with the additional argument `compoundsBackground = background`. How many pathways are significantly enriched after using the measured metabolites as the background?
7. Optional homework! Take some of your own data, use this markdown file as a template, and see how far you can get with processing and visualizing it. Start by changing `sample_meta` to match your samples and experimental data.

## Convert R Markdown to PDF

To convert an R Markdown file into a PDF, R requires an additional program called LaTeX. One simple and lightweight option is TinyTeX, which installs just the components needed for R Markdown and works behind the scenes to generate PDF output.

Installing TinyTeX is optional and can be done later if you are interested in generating PDFs. The code chunk below shows how to install it, but it is tagged with `eval = FALSE` so it will not run automatically. The installation is several hundred megabytes and may take a little time. In some cases, installation issues can occur that require additional troubleshooting, so we recommend doing this on your own time rather than during the workshop.

If you decide to install TinyTeX, simply remove, `eval = FALSE` from the chunk header and run the code. After installation, you should see a “Knit” button at the top of your RStudio window. You can use this button to convert your R Markdown file to PDF.

```
install.packages(c("rmarkdown", "knitr", "tinytex"))
tinytex::install_tinytex()
```

## Further reading

If you would like to continue building your skills in R and data analysis, a great place to start is Roger D. Peng’s open-access (free) book *R Programming for Data Science*: <https://bookdown.org/rdpeng/rprogdatascience/>. It introduces many foundational ideas in R programming and data science in a practical and approachable way.

For topics more specific to genomics and bioinformatics workflows, *Computational Genomics with R* by Altuna Akalin is another excellent resource: <https://compgenomr.github.io/book/>. It covers a range of common bioinformatics tasks, including working with sequencing data, visualization, and downstream analysis using R and Bioconductor.

Looking for inspiration on how to visualize your data in R? Explore the R Graph Gallery: <https://r-graph-gallery.com>

Although this workshop focuses on metabolomics, many of the same principles and tools apply across bioinformatics domains. Bioinformatics is, at its core, biological data science. Most well-written data science resources are directly relevant and can help you continue to grow your skills.

## How to get help

Many common questions and answers: <https://stackoverflow.com> (top Google search hits for error messages and problems with R usually have some links to here).

Need to ask someone for help? Make a SSCCE <https://sscce.org>.

Amazing for debugging code and troubleshooting error messages <https://chatgpt.com>.